

Lekcja 10 - Struktury danych w Pythonie

2026-09-12

Data Engineering Course

Struktury danych w Pythonie

```
list, tuple, set, dict
```

Cel: umieć wybrać dobrą strukturę do konkretnej operacji ETL.

■ TL;DR

Ta lekcja nie jest o teorii dla teorii. To jest lekcja o decyzjach, które mają skutki w runtime.

operacja -> koszt -> struktura danych

Jesli wybierzesz zla strukture, kod moze byc poprawny logicznie, ale bedzie wolny, trudny w review i podatny na bugi.

1. Problem ETL i gdzie tracimy czas
2. list, tuple, set, dict w praktyce
3. Jeden case: dedup + lookup + agregacja
4. Anti-patterny i jak je naprawić
5. Checklista decyzji przed oddaniem homeworku

■ Problem biznesowy

Masz surowe orders z API. Chcesz policzyć revenue per customer i oddać stabilny artefakt.

```
musisz usunac duplikaty  
musisz szybko sprawdzac rekordy po ID  
musisz policzyc agregacje  
musisz utrzymac czytelny kod i testy
```

Jedna struktura nie obsługuje wszystkiego dobrze.

```
orders = [  
    {"order_id": "1001", "customer_id": "C1",  
     "status": "completed", "total_amount": "120.5"},  
    {"order_id": "1002", "customer_id": "C1",  
     "status": "pending", "total_amount": "80.0"},  
    {"order_id": "1001", "customer_id": "C1",  
     "status": "completed", "total_amount": "120.5"}  
]
```

To jest typowy payload: słowniki, duplikaty, stringowe kwoty.

■ list: kiedy używać

list jest dobra, gdy:

- liczy się kolejność,
- idziesz rekord po rekordzie,
- budujesz wynik etapami.

To jest naturalna struktura dla sekwencji accepted records.

■ list: gdzie boli

Nie używaj listy jako głównej struktury do lookupów.

```
if order_id in processed_ids_list
```

To oznacza linear search. Przy dużych danych taki kod zwykle jest głównym powodem spowolnienia.

■ tuple: po co istnieje

tuple jest po to, żeby mieć stabilny, niemutowalny klucz.

Przykład:

```
(customer_id, order_date)
```

Taki klucz dobrze działa w dict przy grupowaniu po wielu polach.

■ tuple: praktyczny pattern

```
count_by_customer_day: dict[tuple[str, str], int]
= {}
key = (customer_id, order_date)
count_by_customer_day[key] =
count_by_customer_day.get(key, 0) + 1
```

To jest prosty i czytelny sposób na composite grain.

■ set: kiedy jest najlepszy

set wybierasz, gdy potrzebujesz:

- unikalności,
- szybkiego membership check,
- deduplikacji po ID.

W ETL to jedna z najczęściej używanych struktur.

■ set: dedup w praktyce

```
seen_ids: set[str] = set()
accepted: list[dict[str, str]] = []

for order in orders:
    order_id = order.get("order_id", "")
    if order_id in seen_ids:
        continue
    seen_ids.add(order_id)
    accepted.append(order)
```

Tu list i set pracuja razem: list trzyma kolejnosc, set pilnuje unikalnosci.

■ dict: kiedy jest najlepszy

dict wybierasz, gdy robisz:

- lookup po kluczu,
- counting,
- grouping,
- mapowanie key -> value.

W Python ETL dict jest główna struktura dla indeksów i agregacji.

dict: agregacja revenue

```
revenue_by_customer: dict[str, float] = {}

for order in accepted:
    if order.get("status", "").lower() != "completed":
        continue
    customer_id = order["customer_id"]
    amount = float(order["total_amount"])
    revenue_by_customer[customer_id] =
    revenue_by_customer.get(customer_id, 0.0) + amount
```

Ta logika jest prosta do review i prosta do testowania.

■ Złożoność i koszt

```
list membership:  $O(n)$   
set membership:  $O(1)$  average  
dict lookup:  $O(1)$  average
```

Interpretacja praktyczna:

- kilka rekordów: różnica mała,
- miliony rekordów: różnica krytyczna.

■ Myslowy benchmark

Masz 5 000 000 order_id i sprawdzasz duplikaty.

```
list -> wiele porownan na kazdy rekord, koszt  
rosnie szybko  
set -> szybkie sprawdzenie, koszt rośnie znacznie  
wolniej
```

Nie chodzi o idealna formule. Chodzi o intuicje kosztu.

■ Anti-pattern 1

Wszystko jako list.

Objaw:

```
lokalnie działa,  
na większym batchu pipeline mocno zwalnia
```

Naprawa:

- dedup i membership do set,
- lookup i agregacje do dict.

■ Anti-pattern 2

set tam, gdzie musi być kolejność biznesowa.

Objaw:

```
niestabilny porządek wyników,  
trudne porównanie outputów między runami
```

Naprawa:

- sekwencje trzymaj w list,
- set używaj tylko jako pomocniczy indeks unikalności.

■ Anti-pattern 3

Nieplanowana mutacja rekordów.

Objaw:

```
jeden etap zmienia dane,  
kolejny etap dostaje nieoczekiwany stan,  
testy przechodzą tylko czasami
```

Naprawa:

- ogranicz mutacje,
- jawnie nazywaj etapy,
- testuj edge case i puste dane.

Czy potrzebuje kolejnosci?

tak -> list

nie -> dalej

Czy potrzebuje unikalnosci lub membership?

tak -> set

nie -> dalej

Czy potrzebuje mapowania key -> value?

tak -> dict

nie -> dalej

Czy potrzebuje immutable composite key?

tak -> tuple

1. Przyjmij input jako list rekordow.
2. Zdefiniuj klucz dedup i trzymaj seen_ids w set.
3. Accepted records trzymaj w list.
4. Agregacje i index trzymaj w dict.
5. Gdy grupujesz po wielu polach, uzyj tuple key.
6. Dodaj testy: puste dane, duplikaty, brakujace pola.

Pytania na code review

1. Dlaczego tu jest list, a nie set?
2. Dlaczego dedup robisz przez set?
3. Gdzie jest granica mutacji danych?
4. Czemu ten klucz jest tuple?
5. Jak testem udowadniasz poprawna decyzje?

```
homework/lesson_10/  
  data_structures_etl.py  
  test_data_structures_etl.py  
  notes.md
```

W notes.md:

- ELI5,
- First Principles,
- odpowiedz Socratic,
- uzasadnienie wyboru struktur.

Definition of done

- [] dedup po order_id jest przez set
- [] lookup i agregacje sa przez dict
- [] klucz zlozony jest tuple
- [] list sluzy do uporządkowanej iteracji
- [] testy pokrywaja edge case
- [] decyzje sa obronione na glos

■ Closing check

Pytanie:

Masz 5 mln order_id i chcesz wykryć duplikaty.
list czy set i dlaczego?

Poprawna odpowiedź:

set, bo membership check jest średnio $O(1)$,
a list robi linear search $O(n)$.